Django Templates & Django Forms — Complete Notes

Django 6.0 | Beginner to Advanced | English + Bangla Explanation

---

Part 1: Django Templates — Complete Notes

Chapter 1: What Are Django Templates?

Django Templates generate dynamic HTML pages by combining static HTML with dynamic data passed from Python views.

বাংলায়: Django Template ব্যবহার করে Python view থেকে পাঠানো data দিয়ে dynamic HTML page তৈরি করা হয়।

A template can contain:

- **Static content:** HTML elements and text.
- **Variables:** Display dynamic data.
- **Tags:** Implement template logic, such as loops and conditions.
- **Filters:** Transform displayed values.
- **Comments:** Add notes that are not included in rendered output.

Example

views.py

```
from django.shortcuts import render


def home(request):
    context = {
        "first_name": "John",
        "last_name": "Doe",
    }
    return render(request, "home.html", context)
```

templates/home.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Home</title>
</head>
<body>
```

```
    <h1>Welcome {{ first_name }} {{ last_name }}</h1>
</body>
</html>
```

Output

```
Welcome John Doe
```

Advantages of Django Templates

- Generate dynamic HTML pages.
- Separate presentation from business logic.
- Reuse layouts through template inheritance.
- Display data retrieved from databases.
- Support loops, conditions, and filters.
- Reduce repeated HTML code.

বাংলায়: Template-এর মাধ্যমে HTML presentation এবং Python business logic আলাদা রাখা যায়। ফলে project maintain করা সহজ হয়।

Chapter 2: Django Template Language (DTL)

The Django Template Language (DTL) is Django's built-in template language.

Django supports multiple template engines, including:

1. Django Template Language (DTL).
2. Jinja2.
3. Custom template engines.

DTL is a good default choice for standard Django applications.

Four Main Components of DTL

Component	Syntax	Purpose
Variables	`{{ variable }}`	Display data
Tags	`{% tag %}`	Add template logic

Filters	`{{ variable|filter }}`	Transform values
Comments	`{# comment #}`	Add non-rendered comments

Important: Django templates are not Python code. Arbitrary Python expressions cannot be executed directly inside `{{ }}`.

বাংলায়: Template-এ variable, tag ও filter ব্যবহার করা যায়; কিন্তু সরাসরি Python-এর যেকোনো expression চালানো যায় না।

Chapter 3: Template Variables

Variables display values passed to a template through its context.

Syntax

```
{{ variable_name }}
```

Example

views.py

```
from django.shortcuts import render


def profile(request):
    context = {
        "name": "Omar",
        "age": 25,
    }
    return render(request, "profile.html", context)
```

profile.html

```
<h1>Name: {{ name }}</h1>
<p>Age: {{ age }}</p>
```

Output

```
Name: Omar
Age: 25
```

Dictionary Lookup

```
{{ user.name }}
{{ user.email }}
```

Object Attribute Lookup

```
{{ product.name }}
{{ product.price }}
```

List Index Lookup

```
{{ students.0 }}
{{ students.1 }}
```

DTL uses dot notation to look up dictionary keys, object attributes, and list indexes.

Important: Dot lookup is not identical to Python attribute access. Django resolves dictionary keys, attributes, and indexes according to its template lookup rules.

বাংলায়: `{{ user.name }}` dictionary key বা object attribute থেকে value দেখাতে পারে। List-এর index-ও dot notation দিয়ে ব্যবহার করা যায়।

Chapter 4: Template Tags

Template tags perform operations during template rendering.

বাংলায়: Template tag দিয়ে condition, loop, URL generation এবং static file loading-এর মতো কাজ করা হয়।

4.1 If Tag

The `if` tag conditionally displays content.

```
{% if user.is_authenticated %}
    <h1>Welcome, {{ user.username }}</h1>
{% else %}
    <h1>Please log in</h1>
{% endif %}
```

4.2 Elif Tag

The `elif` clause checks another condition when previous conditions are false.

```
{% if marks >= 80 %}
    <p>Grade: A+</p>
{% elif marks >= 60 %}
    <p>Grade: A</p>
{% else %}
    <p>Grade: B or below</p>
{% endif %}
```

Note: `elif` is used inside an `if` tag block; it is not a separate standalone tag.

4.3 For Tag

The `for` tag iterates over a sequence.

views.py

```
from django.shortcuts import render


def students_view(request):
    students = ["Rahim", "Karim", "Omar"]

    return render(
        request,
        "students.html",
        {"students": students},
    )
```

students.html

```
<ul>
    {% for student in students %}
        <li>{{ student }}</li>
    {% empty %}
        <li>No students found.</li>
    {% endfor %}
</ul>
```

The `{% empty %}` clause displays content when the sequence is empty.

বাংলায়: List-এ কোনো item না থাকলে `{% empty %}` block-এর content দেখানো হয়।

4.4 Forloop Variables

Variable	Meaning

`forloop.counter`	Current iteration, starting at 1
`forloop.counter0`	Current iteration, starting at 0
`forloop.first`	True on the first iteration
`forloop.last`	True on the last iteration
`forloop.revcounter`	Remaining iterations, starting at 1
`forloop.revcounter0`	Remaining iterations, starting at 0

Example

```
{% for student in students %}
    <p>{{ forloop.counter }}. {{ student }}</p>
{% endfor %}
```

4.5 URL Tag

The `url` tag generates URLs from named URL patterns.

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path("profile/", views.profile, name="profile"),
]
```

template.html

```
<a href="{% url 'profile' %}">My Profile</a>
```

For a URL requiring a parameter:

```
path(
    "product/<int:product_id>/",
    views.product_detail,
    name="product_detail",
)
```

```
<a href="{% url 'product_detail' product.id %}">
    View Product
</a>
```

বাংলায়: `{% url %}` named URL pattern থেকে URL তৈরি করে। URL পরিবর্তন হলেও name একই থাকলে template-এ hardcoded URL পরিবর্তনের প্রয়োজন সাধারণত হয় না।

4.6 Static Tag

The `static` tag generates URLs for static files such as CSS, JavaScript, and images.
Load the tag library first:

```
{% load static %}
```

Then use:

```
<link rel="stylesheet" href="{% static 'css/style.css' %}">
<script src="{% static 'js/app.js' %}"></script>
<img src="{% static 'images/logo.png' %}" alt="Logo">
```

4.7 CSRF Token Tag

The `csrf_token` tag adds a CSRF token to applicable POST forms.

```
<form method="post">
    {% csrf_token %}
    <input type="text" name="username">
    <button type="submit">Submit</button>
</form>
```

It helps protect against Cross-Site Request Forgery (CSRF) attacks.

বাংলায়: Django-এর CSRF protection ব্যবহার করলে unauthorized cross-site request-এর মাধ্যমে user-এর browser দিয়ে অনাকাঙ্ক্ষিত action ঘটানোর ঝুঁকি কমে।

4.8 With Tag

The `with` tag assigns a value to a local template variable.

```
{% with total=products|length %}
    <p>Total products: {{ total }}</p>
{% endwith %}
```

This example counts items in a sequence. Use `length` when the goal is to count items in a list or QuerySet.

4.9 Comment Tag

Single-line comment

```
{# This comment will not appear in the output. #}
```

Multi-line comment

```
{% comment %}
This is a multi-line comment.
It will not be rendered.
{% endcomment %}
```

Chapter 5: Template Filters

Filters transform variable values before displaying them.

Syntax

```
{{ variable|filter_name }}
{{ variable|filter_name:"argument" }}
```

Commonly Used Filters

Filter	Example	Purpose

`lower`	`{{ name|lower }}`	Convert to lowercase
`upper`	`{{ name|upper }}`	Convert to uppercase
`title`	`{{ name|title }}`	Capitalize word beginnings
`capfirst`	`{{ name|capfirst }}`	Capitalize the first character
`length`	`{{ items|length }}`	Return the length
`default`	`{{ name|default:"Guest" }}`	Use fallback for false values
`date`	`{{ created_at|date:"Y-m-d" }}`	Format a date
`time`	`{{ created_at|time:"H:i" }}`	Format a time

`truncatechars`	`{{ text|truncatechars:20 }}`	Truncate text
`join`	`{{ items|join:", " }}`	Join sequence items
`safe`	`{{ content|safe }}`	Mark content as safe HTML
`linebreaks`	`{{ text|linebreaks }}`	Convert line breaks into HTML
`wordcount`	`{{ text|wordcount }}`	Count words
`floatformat`	`{{ price|floatformat:2 }}`	Format a number to decimal places

Examples

Lowercase

```
{{ "DJANGO"|lower }}
```

Output:

```
django
```

Uppercase

```
{{ "django"|upper }}
```

Output:

```
DJANGO
```

Default Value

```
{{ username|default:"Guest" }}
```

If `username` is false or empty, the output is:

```
Guest
```

Date Formatting

```
{{ post.created_at|date:"d M Y" }}
```

Possible output:

```
14 Oct 2026
```

Truncate Text

```
{{ description|truncatechars:20 }}
```

The output is limited to approximately 20 characters, including the truncation marker when needed.

Security Note: Django automatically escapes many variable values in HTML templates. Avoid using `safe` with untrusted user input because it can introduce cross-site scripting (XSS) vulnerabilities.

বাংলায়: `safe` filter ব্যবহার করলে HTML escaping বন্ধ হতে পারে। তাই user-এর দেওয়া untrusted content সরাসরি `safe` করা বিপজ্জনক।

Chapter 6: Template Inheritance

Template inheritance allows multiple pages to share a common layout.

It helps avoid repeating the same HTML structure on every page.

6.1 Base Template

templates/base.html

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <title>{% block title %}My Website{% endblock %}</title>
</head>
<body>
    <header>
        <h1>My Website</h1>
    </header>

    <nav>
        <a href="/">Home</a>
    </nav>

    <main>
        {% block content %}
        {% endblock %}
    </main>

    <footer>
        <p>Copyright 2026</p>
    </footer>
</body>
</html>
```

6.2 Child Template

templates/home.html

```
{% extends "base.html" %}

{% block title %}
    Home Page
{% endblock %}

{% block content %}
    <h2>Welcome to Django</h2>
    <p>This is the home page.</p>
{% endblock %}
```

Important Inheritance Tags

- `{% extends "base.html" %}` — inherits a parent template.
- `{% block name %}` — defines a replaceable section.
- `{% endblock %}` — ends a block.

Rule: In a child template, `{% extends %}` should normally be the first template tag.

বাংলায়: Base template-এ common HTML structure রাখা হয়। Child template শুধু প্রয়োজনীয় block-এর content পরিবর্তন করে।

Chapter 7: Template Configuration

Django template engines are configured through the `TEMPLATES` setting in `settings.py`.

Example Configuration

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [BASE_DIR / "templates"],
        "APP_DIRS": True,
        "OPTIONS": {
            "context_processors": [
                "django.template.context_processors.request",
                "django.contrib.auth.context_processors.auth",
                "django.contrib.messages.context_processors.messages",
            ],
        },
    },
]
```

Important Settings

1. BACKEND

Specifies the template engine.

```
"BACKEND": "django.template.backends.django.DjangoTemplates"
```

2. DIRS

Specifies project-level template directories.

```
"DIRS": [BASE_DIR / "templates"]
```

3. APP_DIRS

When `True`, Django searches for templates inside the `templates/` directories of installed applications.

```
"APP_DIRS": True
```

4. OPTIONS

Contains additional settings for the selected template backend, including context processors and other engine options.

Recommended Folder Structure

```
myproject/
├── manage.py
├── myproject/
│   ├── settings.py
│   └── urls.py
├── templates/
│   ├── base.html
│   └── home.html
└── blog/
    ├── views.py
    ├── urls.py
    └── templates/
        └── blog/
            ├── post_list.html
            └── post_detail.html
```

Using application-specific subdirectories such as `templates/blog/` helps prevent template name conflicts.

বাংলায়: Project-level templates-এর জন্য `DIRS` এবং app-specific templates-এর জন্য `APP_DIRS` ব্যবহার করা যায়।

Chapter 8: Template Context

A context contains the data made available to a template.

In common Django usage, the context is a dictionary of values.

Example

```
from django.shortcuts import render


def dashboard(request):
    context = {
        "username": "Omar",
        "age": 25,
        "is_active": True,
        "skills": ["Python", "Django", "SQL"],
    }
```

```
    return render(request, "dashboard.html", context)
```

dashboard.html

```
<h1>{{ username }}</h1>
<p>Age: {{ age }}</p>

{% if is_active %}
    <p>Account is active.</p>
{% endif %}

<ul>
    {% for skill in skills %}
        <li>{{ skill }}</li>
    {% endfor %}
</ul>
```

Context vs Context Processor

Context: Data passed directly to a template, often from a view.

Context processor: A function that receives a request and returns a dictionary of additional context data.

Context processors are useful for information needed by many templates, such as authentication information or shared site settings.

Example Custom Context Processor

myapp/context_processors.py

```
def site_info(request):
    return {
        "site_name": "My Website",
    }
```

Register it in `settings.py`:

```
"OPTIONS": {
    "context_processors": [
        "django.template.context_processors.request",
        "django.contrib.auth.context_processors.auth",
        "django.contrib.messages.context_processors.messages",
        "myapp.context_processors.site_info",
    ],
}
```

Templates rendered with a request can now access:

```
<h1>{{ site_name }}</h1>
```

Chapter 9: Loading and Rendering Templates

Django provides template-loading functions in `django.template.loader`.

9.1 get_template()

Loads a template and returns a template object.

```
from django.template.loader import get_template

template = get_template("home.html")
html = template.render({"name": "Omar"})
```

When a template cannot be found, Django raises `TemplateDoesNotExist`. Invalid template syntax can raise `TemplateSyntaxError`.

9.2 select_template()

Accepts a list of template names and returns the first template that exists.

```
from django.template.loader import select_template

template = select_template([
    "blog/custom_detail.html",
    "blog/post_detail.html",
])
```

This is useful when a specific page can have a custom template with a default fallback.

9.3 render_to_string()

Loads and renders a template, returning the resulting HTML string.

```
from django.template.loader import render_to_string

html = render_to_string(
    "home.html",
    {"name": "Omar"},
)
```

This is useful when you need rendered HTML as a string rather than an HTTP response.

9.4 render()

The `render()` shortcut is the most common way to render a template from a Django view.

```
from django.shortcuts import render


def home(request):
    return render(
        request,
        "home.html",
        {"name": "Omar"},
    )
```

It combines template rendering with an HTTP response.

Comparison

Function	Purpose	Return Value
`get_template()`	Load a template	Template object
`select_template()`	Load the first existing template from a list	Template object
`render_to_string()`	Render a template	String
`render()`	Render a template for a view response	`HttpResponse`

বাংলায়: সাধারণ Django view-এ HTML render করার জন্য `render()` সবচেয়ে প্রচলিত এবং সুবিধাজনক পদ্ধতি।

Chapter 10: Template Loaders

Template loaders locate template files and load them into Django's template system.

With the usual configuration:

```
"APP_DIRS": True
```

Django can find templates in installed applications' `templates/` directories.

Loaders can also search configured project-level directories and other sources.

You generally do not need to configure loaders manually for a standard Django project.

Chapter 11: Django Template Engines

Django supports different template engines.

11.1 Django Template Language (DTL)

Features:

- Built-in Django template tags and filters.
- Template inheritance.
- Automatic HTML escaping by default.
- Integration with Django applications.

Example:

```
{% if user.is_authenticated %}
    <p>Hello, {{ user.username }}</p>
{% endif %}
```

11.2 Jinja2

Jinja2 is an alternative template engine.

Install it:

```
python -m pip install Jinja2
```

Configure it in `settings.py`:

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.jinja2.Jinja2",
        "DIRS": [BASE_DIR / "jinja2_templates"],
        "APP_DIRS": False,
        "OPTIONS": {
            "environment": "myproject.jinja2.environment",
```

```
        },
    },
]
```

Example `myproject/jinja2.py`:

```
from jinja2 import Environment


def environment(**options):
    env = Environment(**options)
    return env
```

Example Jinja2 template:

```
{% if user %}
    <h1>Hello, {{ user.username }}</h1>
{% endif %}
```

Jinja2 uses similar but different syntax and provides its own filters, tests, and features. Django-specific template tags and filters are not automatically available in Jinja2. Configure equivalent functions or filters when needed.

DTL vs Jinja2

Feature	DTL	Jinja2
Django built-in backend	Yes	No, requires installation
Django template tags	Yes	Not automatically
Template inheritance	Yes	Yes
Filters	Yes	Yes, with different

		details
Calling functions	Restricted	Supports additional callable expressions, subject to configuration
Typical choice	Standard Django applications	When its features or ecosystem are preferred

---

Chapter 12: Template Engine API Components

Django exposes several template-related APIs.

- `django.template.Engine`: Represents an instance of the DTL engine.
- `Template`: Represents a compiled template.
- `Context`: Stores data used during rendering.
- `RequestContext`: Associates rendering with an HTTP request and runs configured context processors.
- Template backend: Adapts an engine to Django's common template API.

For most applications, use `render()` without interacting directly with these lower-level APIs.

Chapter 13: Template Partials in Django 6.0

Django 6.0 introduced support for defining and loading template partials with the Django Template Language backend.

A partial is a reusable fragment of a template.

Define a Partial

```
{% partialdef user_card %}
    <div class="user-card">
        <h2>{{ user.username }}</h2>
        <p>{{ user.email }}</p>
    </div>
{% endpartialdef %}
```

Load the Partial

```
from django.template.loader import get_template

partial = get_template("users.html#user_card")
html = partial.render({"user": user})
```

The returned template object contains the partial's content.

Partials are useful for reusable template fragments. Do not assume this syntax is available in older Django versions.

বাংলায়: Partial দিয়ে বড় template-এর নির্দিষ্ট অংশ আলাদাভাবে reuse করা যায়। Django version অনুযায়ী feature support যাচাই করতে হবে।

Chapter 14: HTML Autoescaping and Template Security

Django's DTL enables HTML autoescaping by default.

For example:

```
{{ user_input }}
```

If `user_input` contains HTML-like text, Django normally escapes it instead of interpreting it as executable HTML.

Security Best Practices

- Keep autoescaping enabled for HTML templates.
- Do not use `safe` with untrusted user input.
- Never allow untrusted users to author templates that your application executes.
- Use `{% csrf_token %}` in applicable POST forms.
- Validate data appropriately before using it in sensitive contexts.
- Use URL reversing with `{% url %}` instead of hardcoding internal URLs.
- Do not expose secrets or sensitive information in template context.
- Apply context-appropriate escaping when generating JavaScript, CSS, URLs, or other non-HTML content.

Important: HTML autoescaping does not automatically make every use of untrusted data safe in every context.

Chapter 15: Common Template Errors

Error 1: TemplateDoesNotExist

Cause: Django cannot find the requested template.

Solutions:

- Check the template filename.
- Check the `DIRS` setting.
- Check whether the application is in `INSTALLED_APPS`.
- Verify the app's `templates/` directory.
- Use the correct template path.

Error 2: TemplateSyntaxError

Cause: Invalid template syntax.

Incorrect:

```
{% if user.is_authenticated %}
    <p>Welcome</p>
```

Correct:

```
{% if user.is_authenticated %}
    <p>Welcome</p>
{% endif %}
```

Error 3: Variable Displays Nothing

Possible causes:

- The variable was not passed in the context.
- The variable name is misspelled.
- A dictionary key or object attribute does not exist.
- The value resolves to an empty string.

Example:

```
return render(
    request,
    "home.html",
    {"username": "Omar"},
)
```

```
{{ username }}
```

Error 4: Static Files Do Not Load

Check:

- `{% load static %}` is present.
- `STATIC_URL` is configured.
- The file exists at the expected location.
- Static-file settings are correct for development or production.

Error 5: Template Logic Is Too Complex

Solution: Move database queries, business logic, and complex calculations into views, models, or other Python code. Keep templates focused on presentation.

Chapter 16: Best Practices

- Use template inheritance with a shared `base.html`.
- Organize templates into application-specific directories.
- Use `{% url %}` for internal links.
- Use `{% load static %}` for static files.
- Use context processors only for genuinely shared data.
- Keep business logic out of templates.
- Keep HTML autoescaping enabled.
- Use clear, descriptive template names.
- Prefer Django's `render()` shortcut for ordinary view rendering.
- Use DTL unless another engine provides a clear advantage.
- Test templates and verify that expected data is displayed.

Chapter 17: Quick Revision

Topic	Syntax or API	Purpose
Variable	`{{ name }}`	Display a value

Tag	`{% if condition %}`	Perform template logic
Filter	`{{ name|upper }}`	Transform a value
Comment	`{# comment #}`	Add a non-rendered comment
Multi-line comment	`{% comment %}...{% endcomment %}`	Add a non-rendered block
Loop	`{% for item in items %}`	Iterate over items
Inheritance	`{% extends "base.html" %}`	Reuse a parent layout
Block	`{% block content %}`	Define a replaceable section
URL	`{% url 'profile' %}`	Reverse a named URL pattern
Static file	`{% static 'css/sty`	Generate a static-file URL

	`le.css' %}`	
CSRF	`{% csrf_token %}`	Render the CSRF token
Render	`render(request, template, context)`	Return a rendered HTTP response
Load template	`get_template()`	Retrieve a template object
Select template	`select_template()`	Choose the first existing template
Render to string	`render_to_string()`	Return rendered HTML as a string
Configuration	`TEMPLATES`	Configure template engines

Chapter 18: Practice Projects

1. Create a home page using template variables.
2. Display students using a `for` loop.
3. Show different messages using `if`, `elif`, and `else`.
4. Format dates and prices using template filters.

5. Create `base.html` and extend it in three pages.
6. Display database products using data passed through a view.
7. Configure a project-level templates directory.
8. Write a custom context processor.
9. Load a template with `get_template()` and render it.
10. Configure Jinja2 as an alternative template backend.
11. Build a reusable template partial using Django 6.0.
12. Test template escaping with harmless HTML-like input.

Conclusion

Django Templates separate the presentation layer from Python application logic. Understanding variables, tags, filters, inheritance, contexts, template loading, configuration, and security helps you build maintainable and dynamic Django web applications.

বাংলায়: Django Template ভালোভাবে শিখলে dynamic webpage, reusable layout এবং database-driven page তৈরি করা সহজ হয়।

Part 2: Django Forms — Complete Notes

Chapter 19: What Are Django Forms?

Django Forms provide a framework for collecting user input, generating HTML form fields, validating submitted data, converting values into Python data types, displaying validation errors, and processing submissions.

বাংলায়: Django Forms ব্যবহার করে user input নেওয়া, validate করা, Python data type-এ convert করা এবং error দেখানোর কাজ সহজ করা যায়।

Common Uses

- User registration and login.
- Contact forms.
- Search and filter forms.
- Profile updates.
- Product creation.
- File uploads.
- Data entry and editing.
- Bulk data entry using formsets.

Advantages of Django Forms

- Reduce the amount of HTML and Python code required.
- Provide built-in field validation.
- Convert input values into appropriate Python types.
- Display validation errors.
- Preserve submitted values when validation fails.
- Support custom validation.
- Integrate with models through `ModelForm`.
- Help implement consistent form handling.

Chapter 20: HTML Forms

An HTML form collects data from users and submits it to a server.

Example

```
<form action="/your-name/" method="post">
    <label for="your_name">Your name:</label>

    <input
        type="text"
        id="your_name"
        name="your_name"
    >

    <button type="submit">Submit</button>
</form>
```

Important HTML Attributes

Attribute	Purpose
`action`	Specifies where the form is submitted
`method`	Specifies the HTTP method

`type`	Defines the input type
`name`	Identifies the submitted value
`id`	Identifies an HTML element
`required`	Enables browser-side required-field validation
`placeholder`	Displays a hint inside an input
`enctype`	Defines how form data is encoded

The HTML `required` attribute improves browser-side validation, but server-side validation is still necessary.

Chapter 21: GET and POST Methods

GET Method

GET is primarily used to retrieve information from a server.

Characteristics:

- Data is generally included in the URL query string.
- URLs can be bookmarked and shared.
- Suitable for searching, filtering, and retrieving information.
- Should not be used for operations that change server state.
- Sensitive data should not be placed in query strings.

Example

```
<form action="/search/" method="get">
    <input type="text" name="q">
    <button type="submit">Search</button>
</form>
```

Example URL:

```
/search/?q=django
```

POST Method

POST is commonly used to submit data for processing or to perform state-changing operations.

Characteristics:

- Data is generally sent in the request body.
- Suitable for registration, updates, and other submissions.
- Django CSRF protection is generally required for applicable POST requests.
- POST does not encrypt data; HTTPS protects data in transit.

Example

```
<form action="/register/" method="post">
    {% csrf_token %}
    <input type="text" name="username">
    <input type="email" name="email">
    <button type="submit">Register</button>
</form>
```

GET vs POST

GET	POST
Primarily retrieves	Submits data for

data	processing
Data generally appears in the URL query string	Data is generally sent in the request body
Suitable for searches	Suitable for data submissions
Should not change server state	Commonly used for state-changing operations
Query data may be stored in browser history	Request body is not normally displayed in the URL

Important: A POST request is not automatically secure merely because its data is absent from the URL. Use HTTPS and appropriate validation and authorization.
বাংলায়: Search ও filter-এর জন্য সাধারণত GET ব্যবহার করা হয়। Registration, update বা অন্য state-changing action-এর জন্য সাধারণত POST ব্যবহার করা হয়।

Chapter 22: How Django Forms Work

Django Forms primarily help developers:

1. Define form fields.
2. Generate HTML form controls.
3. Validate submitted data.
4. Convert input values into Python data types.
5. Display validation errors.
6. Process or save valid data.

Form Processing Flow

```
User
 |
 v
HTML Form
 |
 v
GET or POST Request
 |
 v
Django View
 |
 v
Form Validation
 |
 +---- Valid ----> Process or Save Data
 |
 +---- Invalid --> Display Errors
```

A typical Django form submission follows the POST-Redirect-GET pattern: process the POST request, redirect after success, and display the form again if validation fails.

Chapter 23: Creating a Django Form

Django provides `forms.Form` for creating regular forms.

forms.py

```
from django import forms


class NameForm(forms.Form):
    your_name = forms.CharField(
        label="Your Name",
```

```
        max_length=100,
    )
```

Explanation

- `forms.Form` : Base class for regular forms.
- `NameForm` : Custom form class.
- `your_name` : Field name.
- `CharField` : Accepts text input.
- `label` : Specifies the field label.
- `max_length=100` : Limits the text length to 100 characters.

Django uses the field definition to validate input and generate the corresponding HTML widget.

Chapter 24: Django Form Fields

Field	Purpose
`CharField`	Text input
`EmailField`	Validates email-like input
`IntegerField`	Integer values
`FloatField`	Floating-point values
`DecimalField`	Decimal values
`BooleanField`	Boolean values

`DateField`	Date values
`DateTimeField`	Date and time values
`TimeField`	Time values
`DurationField`	Duration values
`URLField`	URL validation
`ChoiceField`	Selects one option
`MultipleChoiceField`	Selects multiple options
`FileField`	Accepts uploaded files
`ImageField`	Validates image uploads; requires Pillow
`SlugField`	Validates slug-like values
`RegexField`	Validates input

	using a regular expression
`UUIDField`	Validates UUID values

Example

```
from django import forms


class ProductForm(forms.Form):
    name = forms.CharField(max_length=100)

    price = forms.DecimalField(
        max_digits=10,
        decimal_places=2,
    )

    quantity = forms.IntegerField(
        min_value=1,
    )

    available = forms.BooleanField(
        required=False,
    )

    website = forms.URLField(
        required=False,
    )
```

`DecimalField` is commonly preferred for financial values because decimal arithmetic is more suitable for monetary calculations than binary floating-point arithmetic.

Chapter 25: Common Field Arguments

Field arguments control how data is accepted, displayed, and validated.

Example

```
from django import forms


class StudentForm(forms.Form):
    name = forms.CharField(
        label="Student Name",
        min_length=3,
        max_length=50,
        required=True,
        help_text="Enter your full name.",
    )

    age = forms.IntegerField(
        min_value=1,
        max_value=100,
    )

    email = forms.EmailField()
```

Important Arguments

Argument	Purpose
`label`	Defines the field label
`required`	Determines whether a value is required
`initial`	Specifies an initial display value
`help_text`	Provides additional

	instructions
`min_length`	Defines minimum text length
`max_length`	Defines maximum text length
`min_value`	Defines minimum numeric value
`max_value`	Defines maximum numeric value
`widget`	Defines how the field is rendered
`error_messages`	Customizes validation messages
`validators`	Adds validation functions
`disabled`	Prevents ordinary user

	editing of a field

Custom Error Messages

```
from django import forms


class UsernameForm(forms.Form):
    username = forms.CharField(
        min_length=4,
        max_length=20,
        error_messages={
            "required": "Username is required.",
            "min_length": "At least 4 characters are required.",
            "max_length": "Maximum 20 characters are allowed.",
        },
    )
```

Chapter 26: Widgets

A widget controls how a Django form field is rendered as an HTML input element.

Example

```
from django import forms


class ContactForm(forms.Form):
    name = forms.CharField()

    message = forms.CharField(
        widget=forms.Textarea,
    )

    password = forms.CharField(
        widget=forms.PasswordInput,
    )

    birthday = forms.DateField(
        widget=forms.DateInput(
            attrs={"type": "date"},
```

```
        ),
    )
```

Common Widgets

Widget	HTML Representation
`TextInput`	Text input
`EmailInput`	Email input
`NumberInput`	Number input
`PasswordInput`	Password input
`Textarea`	Multiline text area
`CheckboxInput`	Checkbox
`Select`	Dropdown
`RadioSelect`	Radio buttons
`FileInput`	File input
`DateInput`	Date input
`HiddenInput`	Hidden input

Adding HTML Attributes

```
from django import forms


class LoginForm(forms.Form):
    username = forms.CharField(
        widget=forms.TextInput(
            attrs={
                "class": "form-control",
                "placeholder": "Enter username",
            },
        ),
    )

    password = forms.CharField(
        widget=forms.PasswordInput(
            attrs={
                "class": "form-control",
                "placeholder": "Enter password",
            },
        ),
    )
```

The `attrs` dictionary allows you to define HTML attributes such as CSS classes, placeholders, and input IDs.

Important: A widget controls presentation. The associated form field performs data validation.

বাংলায়: Widget ঠিক করে field HTML-এ কেমন দেখাবে; Field ঠিক করে submitted data কীভাবে validate হবে।

Chapter 27: Bound and Unbound Forms

Unbound Form

An unbound form has not been associated with submitted data.

It is commonly used when displaying a form for the first time.

```
from django import forms


class NameForm(forms.Form):
    name = forms.CharField(max_length=100)
```

```
form = NameForm()

print(form.is_bound)
```

Output:

```
False
```

Characteristics:

- Can display initial values.
- Does not represent submitted form data.

Bound Form

A bound form has data associated with it.

```
form = NameForm({
    "name": "Faruk",
})

print(form.is_bound)
print(form.is_valid())
print(form.cleaned_data)
```

Output:

```
True
True
{'name': 'Faruk'}
```

Differences

Unbound Form	**Bound Form**
No data is bound	Data is bound
`is_bound` is `False`	`is_bound` is `True`

Commonly used for initial display	Commonly used to process submissions
Does not contain bound submission data	May contain submitted data

Important: `NameForm({})` is a bound form, even though the dictionary is empty. A bound form can still be invalid.

Chapter 28: The is_valid() Method

The `is_valid()` method runs validation on a bound form.

It returns `True` when all validation succeeds and `False` when validation fails.

Example

```
from django import forms


class StudentForm(forms.Form):
    name = forms.CharField(max_length=50)
    age = forms.IntegerField(min_value=1)


form = StudentForm({
    "name": "Faruk",
    "age": "22",
})

if form.is_valid():
    print("Form is valid")
else:
    print("Form is invalid")
    print(form.errors)
```

Output:

```
Form is valid
```

If `age` is `"abc"`, the form will be invalid because the value cannot be converted into a valid integer.

Chapter 29: The cleaned_data Dictionary

`cleaned_data` contains validated and converted form values.

```
if form.is_valid():
    print(form.cleaned_data["name"])
    print(form.cleaned_data["age"])
```

Output:

```
Faruk
22
```

Common Type Conversions

Field	Python Type
`CharField`	`str`
`IntegerField`	`int`
`FloatField`	`float`
`DecimalField`	`decimal.Decimal`
`BooleanField`	`bool`
`DateField`	`datetime.date`
`DateTimeField`	`datetime.dateti`

	`me`

Rule: Read validated field values from `cleaned_data` after `is_valid()` returns `True`.

বাংলায়: `request.POST`-এর value সাধারণত string আকারে আসে। Django form validation-এর পরে `cleaned_data`-তে প্রয়োজন অনুযায়ী Python type পাওয়া যায়।

Chapter 30: Complete Django Form Project

This example demonstrates a simple name form.

Project Structure

```
myproject/
├── manage.py
├── myproject/
│   ├── settings.py
│   └── urls.py
└── contact/
    ├── forms.py
    ├── views.py
    ├── urls.py
    └── templates/
        └── contact/
            ├── name.html
            └── thanks.html
```

Step 1: Create the App

```
python manage.py startapp contact
```

Add the app to `INSTALLED_APPS` in `settings.py`:

```
INSTALLED_APPS = [
    # Default Django apps...
    "contact",
]
```

Step 2: Create forms.py

```
from django import forms
```

```
class NameForm(forms.Form):
    your_name = forms.CharField(
        label="Your Name",
        min_length=2,
        max_length=100,
    )
```

Step 3: Create views.py

```
from django.shortcuts import render, redirect
from .forms import NameForm


def get_name(request):
    if request.method == "POST":
        form = NameForm(request.POST)

        if form.is_valid():
            name = form.cleaned_data["your_name"]
            request.session["your_name"] = name
            return redirect("thanks")
    else:
        form = NameForm()

    return render(
        request,
        "contact/name.html",
        {"form": form},
    )


def thanks(request):
    name = request.session.get("your_name", "")

    return render(
        request,
        "contact/thanks.html",
        {"name": name},
    )
```

Step 4: Create contact/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path("your-name/", views.get_name, name="get_name"),
    path("thanks/", views.thanks, name="thanks"),
]
```

Step 5: Configure Project URLs

In the project's `urls.py`:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("contact.urls")),
]
```

Step 6: Create name.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Name Form</title>
</head>
<body>
    <h1>Enter Your Name</h1>

    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Submit</button>
    </form>
</body>
</html>
```

Step 7: Create thanks.html

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <title>Thank You</title>
</head>
<body>
    <h1>Thank You!</h1>
    <p>Hello, {{ name }}!</p>
</body>
</html>
```

Step 8: Run the Development Server

```
python manage.py runserver
```

Open:

```
http://127.0.0.1:8000/your-name/
```

This example uses a Django session to carry the name to the success page. In larger applications, choose an appropriate persistence method based on requirements.

Chapter 31: Standard Form Handling Pattern

This is the standard pattern for handling a regular Django form.

```
from django.shortcuts import render, redirect
from .forms import MyForm


def my_view(request):
    if request.method == "POST":
        form = MyForm(request.POST)

        if form.is_valid():
            data = form.cleaned_data

            # Process or save validated data.
            return redirect("success")
    else:
        form = MyForm()

    return render(
        request,
        "my_template.html",
```

```
        {"form": form},
    )
```

Explanation

- `MyForm()` creates an unbound form.
- `MyForm(request.POST)` binds submitted data to the form.
- `is_valid()` runs validation.
- `cleaned_data` provides validated values.
- `redirect()` sends the browser to another URL.
- `render()` returns a response containing a rendered template.

If validation fails, the bound form is rendered again with the submitted values and validation errors.

Chapter 32: CSRF Protection

CSRF stands for Cross-Site Request Forgery.
Django's CSRF protection helps prevent unauthorized cross-site requests from performing actions using a user's browser.

Template Example

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Submit</button>
</form>
```

Important Points

- Include `{% csrf_token %}` in applicable POST forms.
- Django's CSRF middleware is enabled by default in a standard project configuration.
- A CSRF token does not replace authentication or authorization.
- Use HTTPS to protect sensitive data in transit.
- Do not disable CSRF protection as a shortcut to fix form submission errors.

Chapter 33: Contact Form Example

forms.py

```
from django import forms


class ContactForm(forms.Form):
    subject = forms.CharField(max_length=100)

    message = forms.CharField(
        widget=forms.Textarea,
    )

    contact_email = forms.EmailField()

    urgent = forms.BooleanField(
        required=False,
    )
```

views.py

```
from django.shortcuts import render, redirect
from .forms import ContactForm


def contact_view(request):
    if request.method == "POST":
        form = ContactForm(request.POST)

        if form.is_valid():
            subject = form.cleaned_data["subject"]
            message = form.cleaned_data["message"]
            email = form.cleaned_data["contact_email"]
            urgent = form.cleaned_data["urgent"]

            # Save the data or send an email here.
            return redirect("contact_success")
    else:
        form = ContactForm()

    return render(
        request,
        "contact/contact.html",
        {"form": form},
    )
```

```
def contact_success(request):
    return render(request, "contact/success.html")
```

contact.html

```
<h1>Contact Us</h1>

<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Send Message</button>
</form>
```

Remember to define the corresponding URL patterns for `contact_view` and `contact_success`.

Chapter 34: Form Rendering

Form rendering is the process of displaying Django form fields in an HTML template.

Method 1: Render the Form Directly

```
<form method="post">
    {% csrf_token %}
    {{ form }}
    <button type="submit">Submit</button>
</form>
```

Method 2: form.as_p

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Submit</button>
</form>
```

Fields are rendered using paragraph-based markup.

Method 3: form.as_div

```
<form method="post">
    {% csrf_token %}
    {{ form.as_div }}
    <button type="submit">Submit</button>
</form>
```

Fields are rendered using div elements. Check the documentation for your installed Django version.

Method 4: form.as_table

```
<form method="post">
    {% csrf_token %}
    <table>
        {{ form.as_table }}
    </table>
    <button type="submit">Submit</button>
</form>
```

Important Points

- `form.as_p` : Paragraph-based rendering.
- `form.as_div` : Div-based rendering.
- `form.as_table` : Table-based rendering.
- `form.field` : Renders an individual field.
- `form.field.label_tag` : Renders a label.
- `form.field.errors` : Displays field errors.
- `form.non_field_errors` : Displays form-level errors.
- `form.hidden_fields` : Provides hidden fields.
- `form.visible_fields` : Provides visible fields.

Rendering helpers do not create the outer HTML `<form>` element or the submit button. Add those manually.

Chapter 35: Rendering Individual Fields

Individual field rendering provides more control over form layout.

```
<form method="post">
    {% csrf_token %}
    {{ form.non_field_errors }}

    <div>
        {{ form.subject.errors }}
        {{ form.subject.label_tag }}
        {{ form.subject }}
    </div>
```

```
    <div>
        {{ form.message.errors }}
        {{ form.message.label_tag }}
        {{ form.message }}
    </div>

    <div>
        {{ form.contact_email.errors }}
        {{ form.contact_email.label_tag }}
        {{ form.contact_email }}
    </div>

    <div>
        {{ form.urgent.errors }}
        {{ form.urgent.label_tag }}
        {{ form.urgent }}
    </div>

    <button type="submit">Submit</button>
</form>
```

Important BoundField Attributes

- `field.label` : The field's label.
- `field.label_tag` : The HTML label element.
- `field.errors` : Errors for that field.
- `field.help_text` : Additional instructions.
- `field.value` : The value used for rendering.
- `field.html_name` : The HTML input name.
- `field.id_for_label` : The ID associated with the label.
- `field.is_hidden` : Indicates whether the field is hidden.
- `field.field` : The underlying form field instance.

Loop Through Fields

When iterating over a form, each item is a `BoundField` .

```
{% for field in form %}
    <div class="fieldWrapper">
        {{ field.errors }}
        {{ field.label_tag }}
        {{ field }}
```

```
        {% if field.help_text %}
            <p>{{ field.help_text }}</p>
        {% endif %}
    </div>
{% endfor %}
```

Chapter 36: Form Error Handling

Django provides field-level and form-level validation errors.

Example Form

```
from django import forms


class RegisterForm(forms.Form):
    username = forms.CharField(
        min_length=4,
        max_length=20,
    )

    email = forms.EmailField()
```

Invalid Data

```
form = RegisterForm({
    "username": "ab",
    "email": "invalid-email",
})

print(form.is_valid())
print(form.errors)
```

Output:

```
False
```

The form is invalid because the username is too short and the email address does not have a valid format.

Display Field Errors

```
{{ form.username.errors }}
```

Display Non-field Errors

```
{{ form.non_field_errors }}
```

Custom Error Loop

```
{% if form.username.errors %}
    <ul class="errorlist">
        {% for error in form.username.errors %}
            <li>{{ error }}</li>
        {% endfor %}
    </ul>
{% endif %}
```

Displaying the bound form again after validation failure is often the simplest way to preserve user input and show errors.

Chapter 37: Custom Validation

Custom validation allows developers to enforce application-specific rules beyond standard field validation.

A. clean_fieldname()

Use this method to validate an individual field.

```
from django import forms


class RegisterForm(forms.Form):
    username = forms.CharField(
        min_length=4,
        max_length=20,
    )

    def clean_username(self):
        username = self.cleaned_data["username"]

        if username.lower() == "admin":
            raise forms.ValidationError(
                "This username is not allowed."
            )

        return username
```

Django automatically calls `clean_username()` during form validation.

Important points:

- Use `self.cleaned_data` to retrieve the validated field value.
- Raise `forms.ValidationError` when the value violates a rule.
- Return the validated value when it is acceptable.

B. clean()

Use `clean()` for form-level validation, particularly when comparing multiple fields.

```
from django import forms


class RegisterForm(forms.Form):
    password = forms.CharField(
        widget=forms.PasswordInput,
    )

    confirm_password = forms.CharField(
        widget=forms.PasswordInput,
    )

    def clean(self):
        cleaned_data = super().clean()

        password = cleaned_data.get("password")
        confirm_password = cleaned_data.get("confirm_password")

        if (
            password
            and confirm_password
            and password != confirm_password
        ):
            raise forms.ValidationError(
                "Passwords do not match."
            )

        return cleaned_data
```

C. add_error()

Use `add_error()` to associate an error with a specific field.

```
def clean(self):
    cleaned_data = super().clean()
```

```
    password = cleaned_data.get("password")
    confirm_password = cleaned_data.get("confirm_password")

    if (
        password
        and confirm_password
        and password != confirm_password
    ):
        self.add_error(
            "confirm_password",
            "Passwords do not match.",
        )

    return cleaned_data
```

This `clean()` method belongs inside a form class that defines both `password` and `confirm_password`.

D. Validator Functions

A validator can be reused across multiple forms.

```
from django import forms
from django.core.exceptions import ValidationError


def validate_not_admin(value):
    if value.lower() == "admin":
        raise ValidationError(
            "The username 'admin' is not allowed."
        )


class RegisterForm(forms.Form):
    username = forms.CharField(
        validators=[validate_not_admin],
    )
```

Use field-specific cleaning methods for field-specific rules, `clean()` for relationships between fields, and reusable validators for rules shared by multiple fields or forms.

Chapter 38: Form vs ModelForm

Regular Form

A regular `Form` is useful when fields do not directly correspond to a database model or when the form is designed for custom processing.

```
from django import forms


class ProductForm(forms.Form):
    name = forms.CharField(max_length=100)

    price = forms.DecimalField(
        max_digits=10,
        decimal_places=2,
    )
```

Model

```
from django.db import models


class Product(models.Model):
    name = models.CharField(max_length=100)

    price = models.DecimalField(
        max_digits=10,
        decimal_places=2,
    )

    def __str__(self):
        return self.name
```

ModelForm

```
from django import forms
from .models import Product


class ProductModelForm(forms.ModelForm):
    class Meta:
        model = Product
        fields = ["name", "price"]
```

The `Meta` class specifies which model the form uses and which model fields are exposed.

Creating a Product

```
from django.shortcuts import render, redirect
from .forms import ProductModelForm


def create_product(request):
    if request.method == "POST":
        form = ProductModelForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("product_success")
    else:
        form = ProductModelForm()

    return render(
        request,
        "products/create.html",
        {"form": form},
    )
```

Differences

Form	**ModelForm**
Fields are generally defined manually	Fields can be generated from a model
Does not automatically save a model	Can save a model instance
Suitable for custom	Suitable for

input processing	creating and updating model objects
Requires custom persistence logic when saving is needed	Provides a convenient `save()` method

Important ModelForm Methods

save()

```
if form.is_valid():
    product = form.save()
```

save(commit=False)

Creates a model instance without saving it to the database immediately.

```
if form.is_valid():
    product = form.save(commit=False)
    product.name = product.name.strip()
    product.save()
```

If the model has many-to-many fields, call `form.save_m2m()` after saving the instance when those relationships need to be saved.

Updating an Existing Object

```
product = Product.objects.get(pk=1)

form = ProductModelForm(
    request.POST or None,
    instance=product,
)
```

```
    if form.is_valid():
        form.save()
```

In a real view, distinguish GET and POST requests explicitly and handle the case where the requested object does not exist.

Security: Prefer explicit field allowlists such as `fields = ["name", "price"]` instead of exposing every model field.

Migration Commands

After changing model definitions:

```
python manage.py makemigrations
python manage.py migrate
```

Chapter 39: Initial Data

The `initial` argument specifies values displayed when an unbound form is rendered.

```
from django import forms


class ProfileForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()


form = ProfileForm(
    initial={
        "name": "Faruk",
        "email": "faruk@example.com",
    },
)
```

Initial values are useful for:

- Default selections.
- Pre-populating profile forms.
- Showing default dates.
- Displaying suggested values.

Important: `initial` provides display defaults. It does not replace submitted values in a bound form and should not be treated as trusted input.

Chapter 40: File Uploads in Django

File uploads allow users to submit documents, images, resumes, and other files through a web application.

A. Regular Form

forms.py

```
from django import forms


class DocumentForm(forms.Form):
    title = forms.CharField(max_length=100)
    document = forms.FileField()
```

Template

```
<form
    method="post"
    enctype="multipart/form-data"
>
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Upload</button>
</form>
```

View

```
from django.shortcuts import render, redirect
from .forms import DocumentForm


def upload_document(request):
    if request.method == "POST":
        form = DocumentForm(
            request.POST,
            request.FILES,
        )

        if form.is_valid():
            uploaded_file = form.cleaned_data["document"]

            # Process or store the uploaded file.
            # A regular Form does not automatically save it.

            return redirect("upload_success")
```

```
    else:
        form = DocumentForm()

    return render(
        request,
        "documents/upload.html",
        {"form": form},
    )
```

B. Important File Upload Concepts

- `request.POST` : Contains ordinary submitted form data.
- `request.FILES` : Contains uploaded file objects.
- `multipart/form-data` : Required for standard HTML file uploads.
- `FileField` : Accepts uploaded files.
- `ImageField` : Validates image uploads and requires Pillow.
- `MEDIA_ROOT` : Defines the local directory for media storage.
- `MEDIA_URL` : Defines the URL prefix for media files.

C. File Storage Model

```
from django.db import models


class Document(models.Model):
    title = models.CharField(max_length=100)

    file = models.FileField(
        upload_to="documents/",
    )

    def __str__(self):
        return self.title
```

D. ModelForm for File Upload

```
from django import forms
from .models import Document


class DocumentModelForm(forms.ModelForm):
    class Meta:
```

```
        model = Document
        fields = ["title", "file"]
```

View

```
from django.shortcuts import render, redirect
from .forms import DocumentModelForm


def upload_document(request):
    if request.method == "POST":
        form = DocumentModelForm(
            request.POST,
            request.FILES,
        )

        if form.is_valid():
            form.save()
            return redirect("upload_document")
    else:
        form = DocumentModelForm()

    return render(
        request,
        "documents/upload.html",
        {"form": form},
    )
```

When a `ModelForm` is saved, Django uses the model's configured file storage to save the uploaded file and stores the file reference in the model.

E. Media Settings

In `settings.py`:

```
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

During development, media files can be served using:

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    # Your URL patterns...
```

```
]

if settings.DEBUG:
    urlpatterns += static(
        settings.MEDIA_URL,
        document_root=settings.MEDIA_ROOT,
    )
```

This development helper is not intended for production media serving. Use an appropriate web server or storage service in production.

F. File Validation

```
import os
from django import forms
from .models import Document


class DocumentModelForm(forms.ModelForm):
    class Meta:
        model = Document
        fields = ["title", "file"]

    def clean_file(self):
        uploaded_file = self.cleaned_data["file"]

        if uploaded_file.size > 5 * 1024 * 1024:
            raise forms.ValidationError(
                "File size must not exceed 5 MB."
            )

        extension = os.path.splitext(
            uploaded_file.name
        )[1].lower()

        if extension != ".pdf":
            raise forms.ValidationError(
                "Only PDF files are allowed."
            )

        return uploaded_file
```

This example checks file size and extension. These checks alone do not prove that a file contains a valid PDF.

File Upload Security

- Validate file size.
- Validate allowed file types.
- Inspect file content when appropriate.
- Use safe file storage and access permissions.
- Avoid trusting the original filename.
- Consider malware scanning for sensitive applications.
- Avoid serving untrusted uploads as executable content.
- Apply authorization checks before allowing downloads.

বাংলায়: File extension দেখে একা সিদ্ধান্ত নেওয়া যথেষ্ট নয়। File content, size এবং access permission-ও যাচাই করতে হবে।

Chapter 41: Hidden and Visible Fields

Django provides methods for working with hidden and visible fields.

```
<form method="post">
    {% csrf_token %}

    {% for hidden in form.hidden_fields %}
        {{ hidden }}
    {% endfor %}

    {% for field in form.visible_fields %}
        <div>
            {{ field.label_tag }}
            {{ field }}
            {{ field.errors }}
        </div>
    {% endfor %}

    <button type="submit">Submit</button>
</form>
```

Important Points

- `form.hidden_fields` : Returns hidden fields.
- `form.visible_fields` : Returns visible fields.

Security note: Hidden fields are still controlled by the client and can be modified. Always validate hidden values on the server.

Chapter 42: Reusable Form Templates

Custom form templates can help maintain consistent HTML markup across forms.

Example: Reusable Field Rendering

```
{% for field in form %}
    <div class="fieldWrapper">
        {{ field.errors }}
        {{ field.label_tag }}
        {{ field }}

        {% if field.help_text %}
            <p>{{ field.help_text }}</p>
        {% endif %}
    </div>
{% endfor %}
```

Using a Custom Form Template

In Django versions supporting the `Form.template_name` and `Form.render()` interfaces:

```
from django import forms


class MyForm(forms.Form):
    name = forms.CharField(max_length=100)

    template_name = "forms/custom_form.html"
```

A form can be rendered using its configured template or an explicitly supplied template:

```
form = MyForm()
rendered_form = form.render()
```

Ensure the template is accessible through Django's template loader. Check the documentation for your installed Django version when customizing the form renderer.

Chapter 43: HTML5 Browser Validation

Django widgets can render HTML input types that support native browser validation. Examples:

- `EmailField` typically uses an email input.
- `URLField` typically uses a URL input.

- `IntegerField` typically uses a number input.

Disabling Native Browser Validation

```
<form method="post" novalidate>
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Submit</button>
</form>
```

The `novalidate` attribute disables native browser form validation.

Django's server-side validation still runs when the submitted form is processed.

Important: Browser-side validation improves usability, but it must not replace server-side validation.

Chapter 44: Formsets

A formset manages multiple instances of the same form together.

Common Uses

- Adding multiple students.
- Entering multiple products.
- Submitting several contact records.
- Processing multiple rows of data.

A. Create a Formset

forms.py

```
from django import forms
from django.forms import formset_factory


class ItemForm(forms.Form):
    name = forms.CharField(max_length=100)

    quantity = forms.IntegerField(
        min_value=1,
    )


ItemFormSet = formset_factory(
    ItemForm,
    extra=3,
```

```
    max_num=5,
    validate_max=True,
)
```

Here:

- `extra=3` : Displays three extra forms by default.
- `max_num=5` : Sets the maximum number of displayed forms.
- `validate_max=True` : Enforces the maximum during validation.

B. Formset Template

```
<h2>Add Multiple Items</h2>

<form method="post">
    {% csrf_token %}
    {{ formset.management_form }}
    {{ formset.non_form_errors }}

    {% for form in formset %}
        <div>
            {{ form.as_p }}
        </div>
    {% endfor %}

    <button type="submit">Submit All</button>
</form>
```

C. Process a Formset

```
from django.shortcuts import render, redirect
from .forms import ItemFormSet


def item_view(request):
    if request.method == "POST":
        formset = ItemFormSet(request.POST)

        if formset.is_valid():
            items = [
                form.cleaned_data
                for form in formset
                if form.cleaned_data
            ]
```

```
            # Process the validated items.
            return redirect("item_success")
    else:
        formset = ItemFormSet()

    return render(
        request,
        "items/formset.html",
        {"formset": formset},
    )
```

D. Why Is management_form Important?

`formset.management_form` supplies management data such as the number of submitted forms.

Django uses this information to construct and validate the submitted formset. Omitting it from the template normally causes formset validation to fail.

E. Formset Validation

```
if formset.is_valid():
    for form in formset:
        if form.cleaned_data:
            print(form.cleaned_data)
```

Always validate the formset before processing submitted values.

F. Formset vs ModelFormSet

Formset: Manages multiple instances of a regular `Form`.

ModelFormSet: Manages multiple model-backed forms and can create or update model objects. Deletion is available when configured appropriately.

Example:

```
from django.forms import modelformset_factory
from .models import Student


StudentFormSet = modelformset_factory(
    Student,
    fields=["name", "email", "age"],
    extra=3,
)
```

Chapter 45: Django Form Security Best Practices

- Validate user input on the server.
- Include CSRF protection in applicable POST forms.
- Apply authentication and authorization where necessary.
- Use explicit field allowlists in `ModelForm`.
- Use Django's authentication system for password management.
- Validate uploaded file size, type, and content.
- Use HTTPS to protect data in transit.
- Avoid revealing sensitive information in error messages.
- Enforce business rules and database constraints.
- Never assume user input is trustworthy.
- Use Django's ORM or parameterized database operations.
- Validate hidden field values on the server.
- Restrict access to private uploaded files.
- Use appropriate rate limiting for sensitive endpoints.
- Avoid storing plain-text passwords or sensitive secrets.

Chapter 46: Common Django Form Mistakes

Mistake	**Correct Approach**
Processing data without validation	Call `is_valid()` first
Accessing values before validation	Use `cleaned_data` after successful validation

Omitting CSRF protection	Include `{% csrf_token %}` where required
Omitting `request.FILES`	Pass both `request.POST` and `request.FILES` for uploads
Omitting upload encoding	Use `multipart/form-data`
Not displaying validation errors	Re-render the bound form
Expecting a regular `Form` to save a model automatically	Use `ModelForm` or implement persistence
Changing server state	Use an appropriate state-

through GET	changing method, commonly POST
Relying only on browser validation	Validate data on the server
Trusting hidden inputs	Validate hidden values on the server
Exposing every model field	Explicitly specify permitted fields
Omitting formset management data	Render `formset.management_form`
Trusting a file extension alone	Validate file content and storage requirements too
Forgetting migrations after	Run `makemigrations`

model changes	and `migrate`

Chapter 47: Interview Questions and Answers

Q1. What are Django Forms?

Django Forms provide a framework for defining form fields, validating submitted data, converting values into Python types, and processing user input.

Q2. What is the difference between Form and ModelForm?

`Form` generally requires fields to be defined manually. `ModelForm` can generate fields from a model and provides a convenient `save()` method.

Q3. What is a bound form?

A bound form is associated with submitted or otherwise supplied data.

Q4. What is an unbound form?

An unbound form has not been associated with submitted data.

Q5. What does is_valid() do?

It runs form validation and returns `True` when the form passes validation.

Q6. What is cleaned_data?

It is a dictionary containing validated and converted form values.

Q7. What is clean_fieldname()?

It is a method used to perform custom validation on a specific field.

Q8. What is clean()?

It performs form-level validation, often including checks involving multiple fields.

Q9. What is a widget?

A widget controls how a form field is rendered as an HTML input element.

Q10. Why is csrf_token used?

It supplies a token used by Django's CSRF protection mechanism to help prevent cross-site request forgery.

Q11. What is the difference between request.POST and request.FILES?

`request.POST` contains submitted form data, while `request.FILES` contains uploaded files.

Q12. What is the purpose of a Django view?

A view receives an HTTP request, executes application logic, processes input, and returns an HTTP response.

Q13. What is initial?

It specifies initial display values for form fields.

Q14. Why should GET requests not change database state?

GET is intended primarily for retrieving data. State-changing operations should use an appropriate method, commonly POST.

Q15. What is a formset?

A formset manages multiple instances of the same form.

Q16. What is a ModelFormSet?

A ModelFormSet manages multiple model-backed forms and supports operations such as creating and updating model instances.

Q17. What is the purpose of form.errors?

It provides validation errors associated with the form's fields and form-level validation.

Q18. What is the difference between clean() and clean_fieldname()?

`clean_fieldname()` validates an individual field, while `clean()` performs form-level validation.

Q19. What does save(commit=False) do?

It creates a model instance without immediately saving it to the database.

Q20. Why is request.FILES necessary?

It provides access to uploaded files in a request.

Q21. Why is management_form required in a formset?

It provides management data that Django needs to construct and validate the submitted formset.

Q22. What is the difference between field errors and non-field errors?

Field errors are associated with specific fields. Non-field errors apply to the form as a whole.

Chapter 48: Practice Problems

Problem 1: Student Form

Create a form containing:

- Name.

- Email.
- Age between 16 and 60.

Requirements:

- Validate every field.
- Display appropriate errors.
- Preserve input when validation fails.

Problem 2: Custom Username Validation

Create a registration form with these rules:

- Minimum length: 5 characters.
- Maximum length: 20 characters.
- The username `admin` must not be accepted.

Use `clean_username()`.

Problem 3: Contact Form

Create a contact form containing:

- Subject.
- Message.
- Email.
- Optional urgent checkbox.

Requirements:

- Use a `Textarea` widget for the message.
- Validate the email.
- Display field errors.

Problem 4: ModelForm

Create a `Book` model with:

- Title.
- Author.
- Publication year.

Create a `ModelForm` and save a valid book to the database.

Problem 5: File Upload

Create a PDF upload form.

Requirements:

- Set a maximum file size.
- Check the file type.

- Save the uploaded file.
- Handle validation errors.

Problem 6: Password Confirmation

Create a registration form containing:

- Username.
- Password.
- Confirm password.

Use `clean()` to ensure both passwords match. For a real registration system, use Django's authentication tools rather than saving raw passwords yourself.

Problem 7: Product Creation

Create a product form containing:

- Name.
- Price.
- Quantity.
- Availability.

Use `ModelForm` to save the product.

Problem 8: Bulk Student Entry

Create a student formset.

Requirements:

- Display three forms initially.
- Validate every submitted form.
- Display formset-level errors.
- Process only validated data.

Problem 9: Profile Update

Create a profile update page that:

- Loads an existing model instance.
- Displays existing values.
- Validates submitted changes.
- Saves the updated instance.

Problem 10: Custom Form Rendering

Create a form template that:

- Displays each label and input separately.
- Shows field errors.

- Shows help text.
- Uses CSS classes.
- Includes CSRF protection.

Chapter 49: Quick Revision Notes

- `forms.Form` : Creates a regular form.
- `forms.ModelForm` : Creates a model-based form.
- `CharField` : Accepts text.
- `EmailField` : Validates email-like input.
- `IntegerField` : Validates and converts integer data.
- `DecimalField` : Handles decimal values.
- `widget` : Defines the HTML representation of a field.
- `initial` : Defines initial display values.
- `request.POST` : Contains submitted form data.
- `request.FILES` : Contains uploaded files.
- `form.is_valid()` : Runs form validation.
- `form.cleaned_data` : Contains validated Python values.
- `form.errors` : Contains validation errors.
- `form.non_field_errors` : Provides form-level errors.
- `clean_<fieldname>()` : Performs field-specific validation.
- `clean()` : Performs form-level validation.
- `add_error()` : Associates a validation error with a field or the form.
- `form.save()` : Saves a model instance through a ModelForm.
- `form.save(commit=False)` : Creates an unsaved model instance.
- `form.hidden_fields` : Returns hidden fields.
- `form.visible_fields` : Returns visible fields.
- `{% csrf_token %}` : Renders the CSRF token.
- `formset.management_form` : Provides management data for a formset.
- `multipart/form-data` : Enables standard HTML file uploads.
- `MEDIA_ROOT` : Defines the local media storage directory.
- `MEDIA_URL` : Defines the media URL prefix.

Chapter 50: The Most Important Django Form Pattern

Memorize this pattern first:

```
from django.shortcuts import render, redirect
from .forms import MyForm


def my_view(request):
    if request.method == "POST":
        form = MyForm(request.POST)

        if form.is_valid():
            data = form.cleaned_data

            # Process or save validated data.
            return redirect("success")
    else:
        form = MyForm()

    return render(
        request,
        "my_template.html",
        {"form": form},
    )
```

For a File-Upload Form

Bind both ordinary form data and uploaded files:

```
form = MyForm(
    request.POST,
    request.FILES,
)
```

For a ModelForm

Save the model instance only after successful validation:

```
if form.is_valid():
    form.save()
```

For a Formset

Render its management form and validate the formset before processing its data:

```
{{ formset.management_form }}
```

```
if formset.is_valid():
    # Process validated forms.
```

```
        pass
```

Conclusion

To master Django Forms, learn these concepts in order:

1. Form classes and field types.
2. GET and POST requests.
3. Bound and unbound forms.
4. `is_valid()`, `cleaned_data`, and errors.
5. Widgets and form rendering.
6. Custom validation.
7. ModelForm and database operations.
8. File uploads and file validation.
9. Formsets and ModelFormSets.
10. Security and real-world project implementation.

Final Recommendation: Practice by building a registration form, contact form, student management system, product creation page, and document upload system. These projects will help you understand how Django Forms are used in real-world backend development.

বাংলায়: প্রথমে basic form handling, তারপর validation, ModelForm, file upload এবং formset practice করো। এভাবে ধাপে ধাপে শিখলে Django backend development-এর বাস্তব কাজগুলো বুঝতে সুবিধা হবে।